

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE CODE: CSA1418

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:50

Annexure A

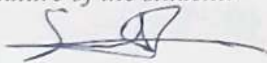
DESIGN TASK

Code of Conduct:

I N. Samanath Reddy (19225083) (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A
DESIGN TASK

Design and implement a recursive descent parser for the grammar:

$S \rightarrow ab \mid ba$

Your design should include:

- (i). Procedure design
- (ii). Matching logic
- (iii). Parsing flow
- (iv). Demonstration for input:ab
- (v). Acceptance

2. Design an unambiguous CFG for arithmetic expressions involving parentheses and multiplication. Further, design a shift-reduce parser and parse:

$(id * id) + id$

Your design should include:

- (i). Grammar
- (ii). Precedence
- (iii). Parsing steps
- (iv). Reduction

3. Design an LL(1) parser for the grammar:

$S \rightarrow A B$

$A \rightarrow a A \mid \epsilon$

$B \rightarrow c$

Your design should include:

- (i). FIRST/FOLLOW
- (ii). Table construction
- (iii). Parsing input:a a c
- (a). Stack
- (b). Actions
- (c). Tree

4. Design and implement a recursive descent parser for the grammar:

$S \rightarrow a S \mid b$

Your design should include:

- (i). Procedures
- (ii). Matching
- (iii). Flow
- (iv). Demonstration for input:a aa b
- (v). Acceptance

⑧ Recursive Descent Parser:

Grammar:

$S \rightarrow ab \mid ba$

⑨ Procedure Design

SC1

{

if (input == 'a')

{

match('a');

match('b');

}

else if (input == 'b')

{

match('b');

match('a');

}

else

error();

}

⑩ matching logic

match(x)

{

if (current-symbol == x)

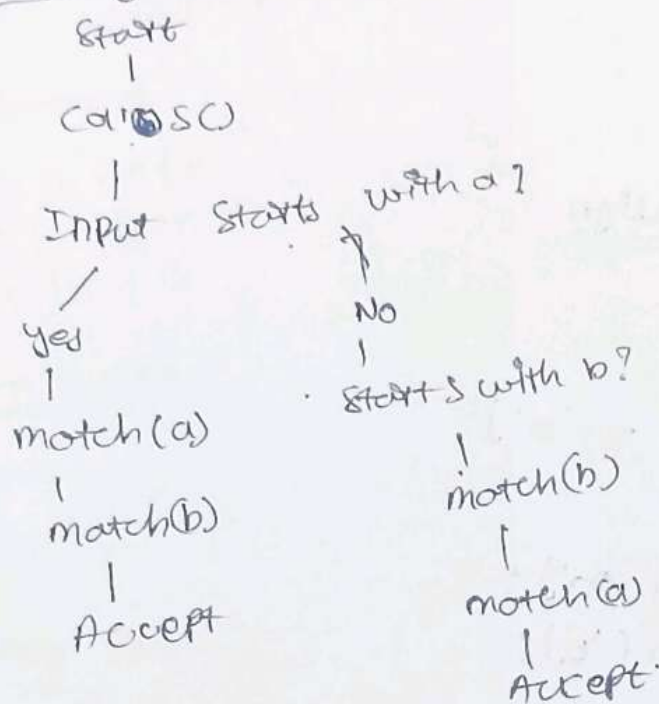
move to next symbol;

else

error();

}

(iii) Passing flow:-



(iv) Demonstration for input: ob

Step	Input	Action
	ob	call SC
1	ob	match(a)
2	b	match(b)
3		Accept
4	EOF	

(v) Acceptance:

Input: ob

Output:

string Accepted.

④ shift Reduce parser

parse: $(id * id) + id$

⑤ Grammar

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

⑥ Precedence:

Highest : $()$

Lowest : $+$

⑦ parsing steps

Stack

\$

\$ (

\$ (id

\$ (F

\$ (T

\$ (E

\$ (E *

\$ (E * id

\$ (E * F

\$ (T

\$ (E

input

$(id * id) + id \$$

$(id * id) + id \$$

$* id) + id \$$

$* id) + id \$$

$* id) + id \$$

$* id) + id \$$

$id) + id \$$

$) + id \$$

$) + id \$$

$) + id \$$

$+ id \$$

Action

shift $()$

shift id

Reduce $id \rightarrow F$

Reduce $F \rightarrow T$

Reduce $T \rightarrow E$

shift $*$

shift id

Reduce $id \rightarrow F$

Reduce $* \rightarrow T$

Reduce $+ \rightarrow E$

shift $)$.

\$ F	id	Reduce $E \rightarrow F$
\$ T	+ id	Reduce $F \rightarrow T$
\$ E	+ id	Reduce $T \rightarrow E$
	id	Shift +
\$ E +		Shift id
\$ E + id	\$	Reduce id $\rightarrow F$
\$ E + F	\$	Reduce $F \rightarrow T$
\$ E + T	\$	Reduce $E + T \rightarrow E$
\$ E	\$	Accept.
\$ E	\$	

Reduction:

- id $\rightarrow F$
- $F \rightarrow T$
- $T \rightarrow E$
- id $\rightarrow F$
- $T \neq F \rightarrow T$
- $T \rightarrow E$
- $(E) \rightarrow F$
- $F \rightarrow T$
- $T \rightarrow E$
- id $\rightarrow F$
- $F \rightarrow T$
- $E + T \rightarrow E$

② LL(1) Parser

Grammar:

$$S \rightarrow AB$$

$$A \rightarrow aA \mid \epsilon$$

$$B \rightarrow c$$

① FIRST and FOLLOW:

$$\text{First}(S) = \{a, c\}$$

$$\text{First}(A) = \{a, \epsilon\}$$

$$\text{First}(B) = \{c\}$$

$$\text{Follow}(S) = \{\#\}$$

$$\text{Follow}(A) = \{c\}$$

$$\text{Follow}(B) = \{\#\}$$

② LL(1) Parsing Table

Non-terminal

S

A

B

$$S \rightarrow AB$$

$$A \rightarrow aA$$

$$S \rightarrow AB$$

$$A \rightarrow \epsilon$$

$$B \rightarrow c$$

③ Parse Input: a a c

④ Stack

#S

#BA

#BAa

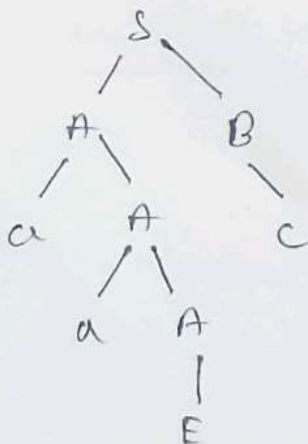
#BA

$\$ B A a$
 $\$ B A$
 $\$ B$
 $\$ C$
 $\$$

(b) Actions:

	input	Action
Start		$S \rightarrow AB$
$\$ S$	$aac\$$	$A \rightarrow aA$
$\$ BA$	$aac\$$	match a
$\$ BAA$	$aac\$$	$A \rightarrow aA$
$\$ BA$	$ac\$$	match a
$\$ BAA$	$ac\$$	$A \rightarrow E$
$\$ BA$	$c\$$	$B \rightarrow C$
$\$ B$	$c\$$	match c
$\$ C$	$\$$	Accept

(c) Parse Tree



29 Recursive Descent Parser

Grammar:

$S \rightarrow aS \mid b$

⑩ Procedures:

S()

{ if (input == 'a')

{ match('a');

S();

}; else if (input == 'b')

{ match('b');

};

else

error();

};

⑪ Matching:

Match(x)

{ if (symbol == x)

Advance();

else

error();

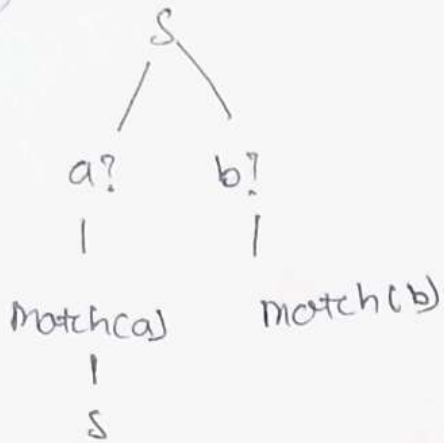
};

E + F
E + T

~~E + F~~
~~E + T~~

Reduce F $\rightarrow$ T

11



10

Demonstration

input
a a b

Step

1

2

3

4

5

input

a a b

a a b

a b

b

EOF

Action.

match(a)

match(a)

match(a)

match(b)

Accept.

9

Acceptance

Input : a a b

Output:

string Accepted.

12

Shift Reduce parser:

Parser:

$id \wedge id + id$

(i) Grammar

$E \rightarrow E + T \mid T$

$T \rightarrow T \wedge F \mid F$

$F \rightarrow id$

(ii) precedence

Highest: $\wedge$

Lowest: $+$

(iii) Parsing steps:

Action

Stack

Input

Shift id

Reduce $id \rightarrow F$

Reduce $F \rightarrow T$

Shift $\wedge$

Shift id

Reduce $id \rightarrow F$

Reduce $T \wedge F \rightarrow T$

Reduce $T \rightarrow E$

Shift $+$

Shift id .

Reduce $id \rightarrow F$

Reduce $F \rightarrow T$

$\$$

$\$ id$

$\$ F$

$\$ T$

$\$ T \wedge$

$\$ T \wedge id$

$\$ T \wedge F$

$\$ T$

$\$ E +$

$\$ E + id$

$\$ E + F$

$\$ E + T$

$id \wedge id + id \$$

$\wedge id + id \$$

$\wedge id + id \$$

$\wedge id + id \$$

$id + id \$$

$+ id \$$

$+ id \$$

$+ id \$$

$id \$$

$\$$

$\$$

$\$$

$\$E$

$\$E$

$\$$

$\$$

Reduce $E \rightarrow T$

Accept.

⑩ Reduction

$id \rightarrow F$

$F \rightarrow T$

$id \rightarrow F$

$T \wedge F \rightarrow T$

$T \rightarrow E$

$id \rightarrow F$

$F \rightarrow T$

$F + T \rightarrow E.$